SKILL-STEP

Skill-Swap Economy Platform

Software Requirements Specification (SRS)

Document Version	2.3
Document Status	Final SRS — Version 2.3 (Supervisor Review Edition)
Project	Computer Information Systems Graduation Project
Institution	University of Jordan — Aqaba Branch
Students	Mohammad Baniyounis | NoorAldeen Amro
Supervisor	Dr. Sufian Al-Khawaldeh
Date	April 2026
Standard	IEEE 830-1998 (Adapted)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, and interface requirements for Skill-Step — a gamified, token-economy-powered online course marketplace. It is the authoritative reference for all development, testing, and acceptance activities and supersedes all prior drafts.

1.2 Scope

Skill-Step is a web-based platform (PHP 8.x / MySQL 8.0+ / HTML5 / CSS3 / JavaScript ES6) that enables users to create and sell short video-based courses, earn virtual tokens through learning and gamification, and convert tokens to real money subject to platform rules. The system is a course marketplace with a token economy and gamification layer — not a peer-to-peer real-time skill-exchange service.

In scope:

- User registration, authentication, and profile management
- Course creation, submission, admin review, and full lifecycle management
- Video lesson upload, server-protected delivery, and server-side progress tracking
- Token-based course purchase with smart escrow mechanism
- Dispute resolution workflow with admin arbitration
- Certificate generation (PDF + QR code) and public QR verification
- Gamification: XP/leveling system, streak bonuses, achievement badges
- Token cash-out with admin approval (simulated payout — see §1.5)
- Admin dashboard: user management, course review, dispute handling, cash-out approvals, analytics
- In-app notification system (email notifications best-effort — see §1.5)
- Quiz subsystem (optional per course; required for certificate in quiz mode)

Out of scope:

- Real-time video conferencing or live tutoring sessions
- Live payment gateway API integration. Cash-out is admin-approved and manually processed externally.
- Mobile native applications (iOS / Android)
- Third-party LMS integration (Moodle, Canvas, etc.)
- Automated KYC, AML, or financial compliance checks
- Horizontal scaling, load balancing, or CDN delivery (single-server prototype)

1.3 Definitions and Abbreviations

Term / Acronym	Definition
Token	Skill-Step virtual currency. Fixed rate: 1 Token = $0.10 USD. Non-transferable between users.
XP (Experience Points)	Points earned by completing lessons or courses. Used to calculate user level.
Level	User rank derived from accumulated XP. Level > 20 required for cash-out eligibility.
Escrow	Temporary hold of tokens paid by a student upon course purchase. Released to creator on reaching the completion threshold.
Release Threshold	Creator-defined minimum course completion percentage (≥ 20%) that triggers escrow release.
Streak	Consecutive calendar days on which a qualifying learning activity occurs. A +500 token bonus is granted every 3rd consecutive active day.
Achievement / Badge	Unlockable award granted when a user's metric reaches a defined condition value.
Cash-Out	Process by which a user converts tokens to USD equivalent. Admin-approved; externally paid manually.
Dispute	A student's formal complaint against a course purchase. Freezes escrow; resolved by admin arbitration.
QR Token	Unique cryptographic string (UUID v4) embedded in a certificate QR code for public verification.
is_active	Boolean field on the enrollments table. Set to 0 when a student wins a dispute, immediately revoking all lesson access.
SELECT FOR UPDATE	A SQL row-level locking statement that prevents concurrent transactions from modifying the same row until the current transaction commits or rolls back.
Idempotent	A property of an operation such that executing it multiple times produces the same result as executing it once.
FR / NFR / BR / UC / WF / RK / AC	Functional Requirement / Non-Functional Req. / Business Rule / Use Case / Workflow / Risk / Acceptance Criterion
ACID	Atomicity, Consistency, Isolation, Durability — database transaction properties

1.4 Stakeholders and User Classes

Actor	Type	Description	Primary Goals
Guest	Primary — Human	Unauthenticated visitor	Browse/search courses; register
Student (Learner)	Primary — Human	Registered user who purchases and completes courses	Learn, earn certificates, gain XP, earn tokens
Creator (Mentor)	Primary — Human	Registered user who creates and sells courses	Publish courses, earn tokens, cash out

Actor	Type	Description	Primary Goals
Admin	Primary — Human	Platform operator with full governance rights	Approve courses, resolve disputes, manage users, approve cash-outs
QR Verifier	Secondary — Human	Any person scanning a certificate QR code	Confirm certificate authenticity
Email Service	External — System	Transactional email provider (e.g., SendGrid, Mailgun)	Best-effort delivery of verification and notification emails
PDF Generator	External — Library	Server-side PDF library (e.g., Dompdf)	Generate downloadable certificate PDFs

1.5 Assumptions and Dependencies

Assumptions:

- Cash-out processing is entirely manual: admin approves in the system, then transfers funds externally. No payment gateway API is integrated.
- The platform is deployed on a single server (single-server prototype deployment). Horizontal scaling and load balancing are out of scope.
- Video files are stored on the server outside the public web directory and served exclusively via PHP proxy.
- Email notifications are best-effort. Delivery is not tracked or retried. In-app notifications (persisted in the notifications table) are the primary reliable notification channel.
- All users access via a modern desktop/mobile web browser (Chrome 110+, Firefox 110+, Edge 110+, Safari 16+).
- The platform operates in English only in this version.
- SSL/HTTPS is configured at the infrastructure level before deployment.
- No real financial credentials (card numbers, bank account details, real PayPal IDs) are used in any demonstration or testing environment.

External Dependencies:

- SMTP email service (e.g., PHPMailer + SendGrid/Mailgun)
- PHP PDF generation library (Dompdf or equivalent) for certificate generation
- PHP QR code generation library for certificate QR codes
- MySQL 8.0+ with InnoDB engine (row-level locking and ACID transactions required)
- PHP 8.x server environment (Apache or Nginx)

2. User Roles and Permissions Matrix

The following matrix defines feature accessibility per actor. All unlisted endpoints are denied by default. Role checks are performed server-side on every protected request (see NFR-24). Admin role does not grant course enrollment or purchasing rights — an admin who also wishes to participate as a learner or creator must hold the respective role in addition.

Feature	Guest	Student	Creator	Admin	QR Verifier
Browse / search courses	Y	Y	Y	Y	Y
View course detail page	Y	Y	Y	Y	N
Register	Y	N	N	N	N
Login / logout	N	Y	Y	Y	N
Reset password	N	Y	Y	Y	N
Edit own profile	N	Y	Y	N	N
Purchase a course	N	Y	Y*	N	N
Watch purchased lessons	N	Y	Y*	N	N
Raise a dispute	N	Y	N	N	N
Earn tokens / XP	N	Y	Y	N	N
Request cash-out	N	Y	Y	N	N
Create / edit own course	N	N	Y	N	N
Submit course for review	N	N	Y	N	N
View creator dashboard	N	N	Y	N	N
Approve / reject course	N	N	N	Y	N
Unpublish active course	N	N	N	Y	N
Resolve disputes	N	N	N	Y	N
Approve / reject cash-out	N	N	N	Y	N
Manage users (suspend/delete)	N	N	N	Y	N
Manage achievement definitions	N	N	N	Y	N
View platform analytics	N	N	N	Y	N
Verify certificate via QR	Y	Y	Y	Y	Y

* A user with creator role may also purchase and consume courses from other creators.

Dual-role note

A single account may hold both student and creator roles simultaneously. Admin role is separately provisioned and cannot be self-assigned.

3. Business Rules

Business rules define platform-wide policies that constrain system behaviour. Each rule is enforced at the application (server) layer, and where feasible also at the database level through constraints. The enforcement column specifies the actual guarantee level for each rule.

3.1 Token Economy Rules

BR-ID	Rule	Enforcement
BR-01	1 Token = $0.10 USD. This rate is fixed and cannot be changed at runtime by any user or admin.	Application + Config
BR-02	New users receive 100 tokens upon successful account activation (email verified). Credited to wallet in the same ACID transaction as account activation.	Application + DB Transaction
BR-03	Tokens are non-transferable between users. Only the platform mints tokens through defined reward events.	Application
BR-04	Token balance is stored exclusively in the wallet table. The users table contains no token balance field. Wallet is the single authoritative source of truth for all balances.	DB design + Application
BR-05	Every token credit or debit must generate a token_ledger entry before the wallet balance is updated, within the same ACID transaction.	Application + DB Transaction

3.2 XP and Leveling Rules

BR-ID	Rule	Enforcement
BR-06	Completing a course (all lessons is_complete = 1) grants XP = floor(price_tokens / 10), minimum 5 XP. Purchasing a course alone grants zero XP.	Application
BR-07	The server setting is_complete = 1 for a lesson (per BR-42) grants 2 XP to the student.	Application
BR-08	XP threshold for level-up: threshold(L) = 1000 + (L − 1) × 200. Level 1→2 requires 1,000 XP; Level 2→3 requires 1,200 XP, and so on.	Application
BR-09	On level-up, XP overflow carries forward. If a user has 1,250 XP and the threshold is 1,000, new XP = 250. Overflow is never discarded.	Application
BR-10	Creators earn 10 XP for each student who completes their course.	Application

Role storage design note

The roles SET('student','creator','admin') type was chosen for prototype implementation simplicity. A production-grade system would use a normalised roles + user_roles junction table for scalable role assignment, future roles (e.g., moderator, finance-admin), and granular permission control.

3.3 Streak Rules

BR-ID	Rule	Enforcement
BR-11	A qualifying activity is: the server sets is_complete = 1 for at least one lesson on a given calendar day. The event is server-triggered, not client-reported.	Application
BR-12	The streak counter (streak_days) increments by 1 each calendar day a qualifying activity occurs. Check: today = last_streak_date + exactly 1 calendar day.	Application
BR-13	If gap > 1 calendar day, streak_days resets to 1 (today is itself an active day).	Application
BR-14	A streak bonus of +100 tokens is granted when streak_days mod 3 == 0. Maximum one bonus per calendar day per user.	Application
BR-15	last_streak_date is updated to today after every qualifying activity check, regardless of whether a bonus was granted.	Application

3.4 Daily Engagement Reward Rules

BR-ID	Rule	Enforcement
BR-16	Daily login reward: +5 tokens granted on the first login of any calendar day. Gated by comparing date(last_login) to today.	Application
BR-17	Daily engagement reward: +10 tokens granted for the first server-confirmed lesson completion on any calendar day. Gated by comparing date(last_streak_date) to today.	Application
BR-18	Both daily rewards are atomic: the reward check and grant execute within the same SELECT FOR UPDATE + ACID transaction as the triggering event to prevent concurrent duplicate grants (BR-46).	Application + DB Transaction

3.5 Escrow Rules

BR-ID	Rule	Enforcement
BR-19	Tokens are placed in escrow (status = 'held') immediately upon course purchase within the same ACID transaction as the wallet deduction.	Application + DB Transaction
BR-20	The minimum release_threshold is 20%. Values below 20% are rejected by both a DB CHECK constraint and server-side validation.	DB CHECK + Application
BR-21	Escrow is released automatically when overall_completion_pct >= release_threshold AND escrow.status = 'held'. Creator wallet is credited in the same ACID transaction.	Application + DB Transaction
BR-22	Once escrow.status = 'frozen' (due to a dispute), no automatic release is possible. Admin must resolve the dispute first.	Application
BR-23	Escrow cannot be released more than once. The system checks status = 'held' using SELECT FOR UPDATE transactional state validation before any release attempt is executed (BR-46).	Application + Transactional state validation (SELECT FOR UPDATE)

3.6 Dispute Rules

BR-ID	Rule	Enforcement
BR-24	A student may raise at most one dispute per purchased course. Enforced by UNIQUE constraint on escrow_id in the disputes table. Since there is exactly one escrow record per enrollment, this guarantees one dispute per enrollment.	DB UNIQUE + Application
BR-25	Disputes may only be raised while escrow.status = 'held'. A student cannot raise a dispute after escrow has been released.	Application
BR-26	A dispute must be raised within 30 calendar days of enrolled_at. Disputes outside this window are rejected with a descriptive error.	Application
BR-27	Dispute resolved in favour of student: escrow.status → 'cancelled'; full token refund credited to student wallet; payment.status → 'cancelled'; enrollment.is_active → 0 (lesson access revoked immediately); both parties notified.	Application + DB Transaction
BR-28	Dispute resolved in favour of creator: escrow.status → 'released'; payment.status → 'released'; tokens credited to creator wallet; enrollment.is_active remains 1; both parties notified.	Application + DB Transaction
BR-29	Partial (prorated) refunds are not supported. Resolution is binary: full refund to student or full release to creator.	Application

3.7 Cash-Out Rules

BR-ID	Rule	Enforcement
BR-30	User level must be strictly greater than 20 to submit a cash-out request.	Application
BR-31	Minimum cash-out request is 100 tokens. Requests below this are rejected.	Application + DB CHECK
BR-32	Tokens are deducted from wallet immediately upon submission (status = 'pending'). If admin rejects, tokens are restored and a Refund ledger entry is created in the same ACID transaction as the status update.	Application + DB Transaction
BR-33	Platform commission is 7% of USD equivalent. net_payout = (tokens × $0.10) × 0.93.	Application
BR-34	The platform records the cash-out request and admin decision only. Actual external payment is performed manually by admin outside the system.	Application
BR-35	A user may have at most 1 pending cash-out request at any time. A second submission while one is pending is rejected.	Application

3.8 Certificate Rules

BR-ID	Rule	Enforcement
BR-36	Default certificate mode (has_quiz = 0): certificate is issued when all lessons have is_complete = 1 AND enrollment.is_active = 1. Revoked enrollments never trigger certificate issuance.	Application

BR-ID	Rule	Enforcement
BR-37	Quiz mode (has_quiz = 1): certificate issued only when 100% video completion AND quiz_attempts record has passed = 1 AND enrollment.is_active = 1.	Application
BR-38	A student receives at most one certificate per course: enforced by UNIQUE(user_id, course_id) in the certificates table.	DB UNIQUE
BR-39	Each certificate receives a unique UUID v4 as qr_token. Never reused or reassigned.	Application
BR-40	The public certificate verification page (/verify/{qr_token}) is accessible without login. It exposes only student full_name, course title, and issued_at date.	Application

3.9 Progress Integrity Rules

These rules define how the server validates that reported video progress is genuine and cannot be manipulated by client-side injection of false completion values.

BR-ID	Rule	Enforcement
BR-41	Progress is tracked by the server accumulating the total unique video timeline coverage reported over all playback sessions for a given (student_id, lesson_id) pair. Completion is based on cumulative unique-interval coverage, not a single client-submitted percentage.	Application
BR-42	Lesson completion (is_complete = 1) is set by the server only when server-side accumulated unique coverage reaches >= 80% of lesson.duration_seconds. The client cannot set or override this flag.	Application
BR-43	Large single-report jumps in watched_pct (advances > 15% per 10-second report window) are discarded as suspicious. The server accumulates only plausible sequential intervals.	Application
BR-44	Repeated playback of the same video segment does not accumulate toward the 80% threshold beyond the first time that segment is covered. Replaying a segment adds zero additional coverage.	Application
BR-45	Simultaneous multi-device/multi-tab playback is permitted, but only one session's progress interval is recorded per report cycle. Duplicate coverage from concurrent sessions is not accumulated.	Application

3.10 Transaction Isolation and Concurrency Rules

These rules define the mandatory concurrency control strategy for all financial and reward operations. They address the risk of double-spending, double-crediting, and race conditions under concurrent requests.

BR-ID	Rule	Enforcement
BR-46	All financial operations (wallet deduction/credit, escrow release, ledger entry, daily reward grant, escrow freeze) must acquire a row-level lock using SELECT ... FOR UPDATE on the relevant wallet and escrow rows	Application + MySQL InnoDB row-level locking

BR-ID	Rule	Enforcement
	before any modification. This prevents concurrent transactions from reading stale balances or states.	
BR-47	Wallet deductions must use a conditional atomic UPDATE statement: UPDATE wallet SET token_balance = token_balance − X WHERE user_id = ? AND token_balance >= X The application must verify that exactly 1 row was affected. If 0 rows affected, the deduction is rejected and the transaction is rolled back — regardless of any prior balance check. This prevents race conditions that bypass application-layer balance validation.	Application + Conditional UPDATE (affected rows check)
BR-48	Escrow is the authoritative source of truth for payment lifecycle state. payment.status must be updated to mirror escrow.status in the same ACID transaction as every escrow status change. The two must never diverge. No code path may update escrow.status without simultaneously updating payment.status to the corresponding value per the mapping: held→pending, released→released, frozen→disputed, cancelled→cancelled.	Application + DB Transaction
BR-49	All transactional endpoints (course purchase, cash-out submission, dispute creation) must be idempotent under retry. Idempotency is guaranteed by: (a) DB-level UNIQUE constraints that reject duplicate records, (b) row-level locking (BR-46) that serialises concurrent attempts, and (c) application-layer checks on existing state before beginning any transaction. A duplicate or retried request returns the same logical outcome without executing a second transaction.	Application + DB UNIQUE + SELECT FOR UPDATE

3.11 Data Retention and Audit Integrity Rules

These rules define which records may never be deleted and how audit trail integrity is preserved throughout the system lifetime.

BR-ID	Rule	Enforcement
BR-50	Core transactional records — token_ledger, payments, escrow, disputes, certificates, and cash_out_requests — must never be physically deleted from the database, even if the associated user account is deleted or suspended. These records constitute the immutable audit trail of the platform.	Application + DB FK RESTRICT on token_ledger.user_id
BR-51	User deletion (if implemented) must cascade only to non-financial records (e.g., sessions, notifications, progress). All financial and transactional records must be retained with user_id intact. The token_ledger.user_id FK uses ON DELETE RESTRICT explicitly to enforce this at the database level.	DB FK ON DELETE RESTRICT + Application
BR-52	Achievement evaluation (WF-3.7) must be event-driven: the system evaluates only achievements whose condition_type matches the event that just occurred (e.g., after course completion, only check 'courses_completed' and 'tokens_earned' achievements). A full-table scan of all achievements on every event is prohibited in production-scale implementations.	Application (event-scoped query)

3.12 Quiz Answer Integrity Rules

BR-ID	Rule	Enforcement
BR-53	Each question in the quiz must have exactly one choice with is_correct = 1. This is enforced at two levels: (a) application-side validation on quiz creation and edit, and (b) a database-level CHECK or trigger that rejects saving a question if the count of correct choices is not exactly 1. At minimum, the application must reject the quiz submission if this condition is violated.	Application validation + DB trigger or CHECK
BR-54	A quiz must have at least 3 questions to be submitted for use. This is enforced application-side on course creation and edit. A course with has_quiz = 1 and fewer than 3 quiz_questions records cannot be submitted for admin review.	Application

4. System Workflows

Each workflow defines the step-by-step behaviour of a specific process. Each layer in this document serves a distinct purpose: Business Rules = policy, Workflows = step sequences, Functional Requirements = what the system must do, Use Cases = actor interaction narratives.

WF-3.1 Course Purchase and Escrow Initialisation

1. User logs in and navigates to an active course detail page.
2. User clicks 'Unlock Course'.
3. Server performs validation (all conditions must pass):
   - course.status = 'active'
   - Logged-in user is NOT the course creator
   - No existing enrollment for (student_id, course_id)
   - wallet.token_balance >= course.price_tokens (preliminary check)
4. If any validation fails: return descriptive error. Abort. No records created.
5. All validations pass — execute single ACID transaction with SELECT FOR UPDATE (BR-46, BR-47, BR-49):
   - SELECT wallet FOR UPDATE on student row to acquire row-level lock
   - Execute conditional UPDATE: token_balance = token_balance − price_tokens WHERE user_id = ? AND token_balance >= price_tokens
   - Verify 1 row affected — if 0: roll back and return 'Insufficient balance'
   - Create Enrollment record (is_active = 1)
   - Create Payment record (status = 'pending')
   - Create Escrow record (status = 'held', amount = price_tokens, creator_id = course.creator_id)
   - Create Token Ledger entry (action_type = 'Purchase', amount = −price_tokens, reference_type = 'payment', reference_id = payment_id)
6. Student granted access to all course lessons (is_active = 1 verified on every lesson request).
7. In-app notification created for student.

WF-3.2 Progress Tracking and Escrow Release

1. Student opens lesson page. PHP proxy checks: valid session AND enrollment exists AND enrollment.is_active = 1. Returns 403 if any check fails.
2. Video player posts watched_pct to server every 10 seconds.
3. Server validates: value in 0–100 range; increment since last report is <= 15% (BR-43). Suspicious increments discarded.
4. Server accumulates unique coverage intervals for this (student_id, lesson_id). Replayed segments not re-accumulated (BR-44).
5. When server-accumulated unique coverage >= 80% of duration_seconds AND is_complete = 0: server sets is_complete = 1; records completed_at; awards 2 XP (BR-07); evaluates daily engagement reward within SELECT FOR UPDATE transaction (BR-18); triggers streak check (WF-3.6).

6. System recalculates overall completion: COUNT(is_complete=1) / total_lessons × 100.
7. If overall_completion >= release_threshold AND escrow.status = 'held': ACID transaction with SELECT FOR UPDATE on escrow row (BR-46, BR-23) — update escrow.status → 'released'; update payment.status → 'released' (BR-48); credit creator wallet; create Token Ledger entry (action_type = 'Escrow_Release', reference_type = 'escrow').
8. If overall_completion = 100% AND enrollment.is_active = 1: trigger WF-3.5 (Certificate Issuance).
9. If escrow.status = 'frozen': skip step 7. No automatic release until admin resolves dispute (WF-3.3).

WF-3.3 Dispute Handling

1. Student clicks 'Raise Dispute' on course page.
2. Server validates eligibility:
   - ↳ escrow.status = 'held' (BR-25)
   - ↳ No existing dispute for this escrow_id (BR-24)
   - ↳ today <= enrolled_at + 30 days (BR-26)
3. If validation fails: return specific reason. Abort.
4. ACID transaction with SELECT FOR UPDATE on escrow row (BR-46): update escrow.status → 'frozen'; update payment.status → 'disputed' (BR-48); create Dispute record (resolution = 'pending', raised_by = student_id).
5. Create in-app notifications for admin, student, and creator.
6. Admin reviews: dispute reason, course content, student progress evidence.
7. Admin resolves:
   - ↳ Creator wins: ACID transaction — escrow.status → 'released'; payment.status → 'released' (BR-48); credit creator wallet; log Escrow_Release ledger entry. enrollment.is_active remains 1.
   - ↳ Student wins: ACID transaction — escrow.status → 'cancelled'; payment.status → 'cancelled' (BR-48); refund student wallet; log Refund ledger entry (reference_type = 'dispute'); enrollment.is_active → 0. Certificate cannot be issued for this enrollment.
8. Create in-app notifications for both parties. dispute.resolved_at and admin_id recorded.

WF-3.4 Cash-Out (Token to Real Money)

1. User navigates to Cash-Out page.
2. Server validates:
   - ↳ user.level > 20 (BR-30)
   - ↳ amount_tokens >= 100 (BR-31)
   - ↳ wallet.token_balance >= amount_tokens (preliminary check)
   - ↳ No existing pending cash-out request for this user (BR-35)
3. User selects payment method and enters masked account identifier only. Full financial credentials are never accepted or stored (BR-50).
4. System displays preview: usd_equivalent, platform_commission (7%), net_payout.
5. User confirms. ACID transaction with SELECT FOR UPDATE on wallet row + conditional UPDATE per BR-47:

↳ UPDATE wallet SET token_balance = token_balance − X WHERE user_id = ? AND token_balance >= X

↳ Verify 1 row affected — if 0: roll back and return 'Insufficient balance'

↳ Create cash_out_requests record (pending)

↳ Create Token Ledger entry (action_type = 'Cash_Out', reference_type = 'cashout')

6. In-app notification created for admin.
7. Admin reviews and acts:

↳ Approve: status → 'approved'; processed_at set; notify user. External payment processed manually.

↳ Reject (mandatory reason): ACID transaction — restore amount_tokens to wallet (unconditional credit); create Refund ledger entry; status → 'rejected'; notify user with reason.

WF-3.5 Certificate Issuance (Default and Quiz Modes)

1. Triggered when overall completion = 100% AND enrollment.is_active = 1 (WF-3.2 step 8).
2. Check UNIQUE(user_id, course_id) in certificates. If already exists: abort silently.
3. Check course.has_quiz flag:

↳ has_quiz = 0: proceed to step 5.

↳ has_quiz = 1: check quiz_attempts for a record where user_id matches AND passed = 1.

4. If quiz mode and no passing attempt: notify student. Student uses WF-3.5a to take/retry quiz.
5. Certificate issuance: generate UUID v4 qr_token; create certificates record; generate PDF; store PDF; update certificates.pdf_path.
6. Create in-app notification for student.

WF-3.5a Quiz Attempt Sub-Workflow

1. Student with 100% video completion clicks 'Take Quiz'. enrollment.is_active = 1 verified.
2. Server confirms 100% completion. Rejects attempt if any lesson incomplete.
3. attempt_no = COUNT(prior attempts for this user/quiz) + 1. Questions presented (randomised if quiz.randomize_questions = 1).
4. Student submits. Server scores: score = (correct / total) × 100. Creates quiz_attempts record (score, passed, attempt_no, taken_at).
5. If passed = 1: trigger WF-3.5 from step 5. If passed = 0: notify student of score; option to retry.

WF-3.6 Streak Bonus

1. Triggered after server sets is_complete = 1 for any lesson (qualifying activity per BR-11).
2. If last_streak_date IS NULL: streak_days = 1; last_streak_date = today. Go to step 6.
3. If today = last_streak_date + 1 day: increment streak_days by 1.
4. If gap > 1 day: reset streak_days = 1.
5. Update last_streak_date = today.

6. If streak_days mod 3 == 0: ACID transaction with SELECT FOR UPDATE on wallet row (BR-46) — credit +100 tokens; create Streak_Bonus ledger entry; create in-app notification.

WF-3.7 Achievement Unlock (Event-Driven)

1. Triggered after any action that may affect an achievement condition.
2. System identifies the triggering event type and queries ONLY achievements whose condition_type matches that event (BR-52):
   - ↳ After lesson completion → evaluate 'courses_completed', 'streak_days', 'tokens_earned'
   - ↳ After level-up → evaluate 'level_reached'
   - ↳ After course published → evaluate 'courses_created'
3. Filter further to achievements where no user_achievements record exists for this user.
4. For each candidate: evaluate condition_type against current user metric.
5. If metric >= condition_value: create user_achievements record.
6. If achievement.token_reward > 0: ACID transaction with SELECT FOR UPDATE on wallet — credit tokens; create Achievement_Reward ledger entry (reference_type = 'achievement', reference_id = achievement_id).
7. Create in-app notification for user.

5. Functional Requirements

Each FR specifies what the system must do. Behaviour detail and step sequences are defined in Section 4 (Workflows) and Section 3 (Business Rules). FRs cross-reference those sections rather than repeating full sequences here.

5.1 User Management

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-01	User Registration	Guest submits registration form	Account created (pending_verification); verification email sent; login blocked until verified	BR-02, UC-01
FR-02	Email Verification	User clicks verification link	Account activated; 100-token bonus credited to wallet in ACID transaction; session started	BR-02, UC-01
FR-03	User Login	Registered user submits valid credentials	Session created; last_login updated; daily login reward evaluated (BR-16)	BR-16, UC-02
FR-04	Login Failure Throttle	5 incorrect passwords within 10 minutes	Account locked 30 minutes; lockout_until set; lockout notification sent	NFR-11, UC-02
FR-05	Password Reset	User submits 'Forgot Password'	If found: single-use UUID reset token (60-min expiry) emailed. If not: generic message — no enumeration.	UC-03
FR-06	Profile Management	User edits bio, avatar, or display name	Changes saved. Avatar: max 2 MB, jpg/png only, MIME-checked server-side.	UC-04
FR-07	Account Suspension	Admin suspends a user	User immediately logged out; all sessions invalidated; login blocked.	UC-16

5.2 Course Management

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-08	Course Creation	Creator submits course with all mandatory fields	Course saved with status = 'pending_review'; admin notified in-app.	BR-20, UC-14
FR-09	Submission Validation	Creator submits with missing fields or threshold < 20%	Rejected with field-level error messages; course remains in draft.	BR-20
FR-10	Admin Course Approval	Admin opens pending course in review queue	Admin approves (→ 'active') or rejects (→ 'rejected', mandatory reason); creator notified.	UC-15

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-11	Course Re-submission	Creator edits rejected course and resubmits	Status → 'pending_review'; prior rejection reason visible to creator.	—
FR-12	Edit Active Course	Creator edits an active course	Status → 'pending_review'; enrolled students retain access; new enrollments disabled.	UC-10
FR-13	Admin Unpublish Course	Admin unpublishes an active course	Course status → 'rejected'; new enrollments blocked; enrolled students retain access; creator notified.	UC-11
FR-14	Lesson Video Upload	Creator uploads video file	Validated: mp4/webm/mkv only, max 500 MB, MIME-checked. UUID-renamed; stored outside web root.	NFR-06

5.3 Enrollment and Purchase

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-15	Course Purchase	Student clicks 'Unlock Course' on active course they do not own	WF-3.1: SELECT FOR UPDATE + conditional UPDATE + ACID transaction creates enrollment (is_active=1), payment, escrow, ledger entry. Student granted access.	BR-04, BR-19, BR-46, BR-47, BR-49, WF-3.1, AC-03
FR-16	Duplicate Purchase Prevention	Student clicks 'Unlock Course' on owned course	'Already enrolled' error. No deduction. DB UNIQUE(student_id, course_id) enforces at DB level.	DB UNIQUE, AC-04
FR-17	Self-Purchase Prevention	Creator attempts to purchase own course	'You cannot purchase your own course.' No transaction.	—
FR-18	Insufficient Balance Prevention	Student's balance < course price	'Insufficient token balance.' Conditional UPDATE returns 0 rows; transaction rolled back. Wallet unchanged.	BR-04, BR-47

5.4 Learning and Progress

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-19	Video Lesson Access Control	Any user requests a lesson video URL	PHP proxy requires: valid session AND enrollment exists AND enrollment.is_active = 1. Returns 403 if any condition	BR-27, WF-3.2, AC-05

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
			fails. Direct URL never accessible.	
FR-20	Progress Tracking	Video player reports watched_pct every 10 seconds	Server accumulates unique coverage per BR-41–45. is_complete = 1 set only when accumulated coverage >= 80%. Client cannot override. Reward grants wrapped in SELECT FOR UPDATE (BR-18, BR-46).	BR-41–45, BR-46, WF-3.2, AC-06
FR-21	Escrow Release on Threshold	Overall course completion reaches release_threshold	If escrow.status = 'held': SELECT FOR UPDATE + ACID transaction releases escrow, updates payment.status (BR-48), credits creator, logs ledger entry.	BR-21, BR-23, BR-46, BR-48, WF-3.2, AC-07
FR-22	Course Completion Detection	All lessons is_complete = 1 AND enrollment.is_active = 1	Awards XP per BR-06; triggers WF-3.5. If is_active = 0: certificate NOT issued.	BR-06, BR-36, BR-37, WF-3.5, AC-11

5.5 Dispute Management

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-23	Raise Dispute	Student raises dispute (escrow = 'held', within 30 days, no prior dispute)	SELECT FOR UPDATE on escrow row; escrow.status → 'frozen'; payment.status → 'disputed' (BR-48); dispute record created; in-app notifications sent. Per WF-3.3.	BR-24–26, BR-46, BR-48, WF-3.3, AC-09
FR-24	Dispute Eligibility Check	Student attempts ineligible dispute	Specific rejection reason returned. Escrow and payment unchanged.	BR-24, BR-25, BR-26, AC-09
FR-25	Admin Dispute Resolution	Admin resolves dispute	Per WF-3.3: escrow, payment, and enrollment.is_active updated atomically in same transaction (BR-48). Both parties notified.	BR-27–29, BR-46, BR-48, WF-3.3

5.6 Gamification

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-26	Streak Bonus	Server sets is_complete = 1; streak check triggered	streak_days updated per BR-12/13. If mod 3 == 0: +100 tokens via SELECT FOR	BR-11–15, BR-46, WF-3.6, AC-08

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
			UPDATE transaction; ledger entry; in-app notification.	
FR-27	XP and Level-Up	Lesson or course completed by server	XP incremented per BR-06/07. Level-up if XP >= threshold(level); overflow carried forward (BR-09); in-app notification.	BR-06–09
FR-28	Achievement Unlock	Any action affecting an achievement condition	Event-driven check per WF-3.7 and BR-52 (no full-table scan). Token reward via SELECT FOR UPDATE if applicable.	BR-46, BR-52, WF-3.7
FR-29	Admin Achievement Management	Admin creates, modifies, or deactivates achievement	Saved to achievements table. Deactivated (is_active = 0) no longer triggers. Existing earned records preserved.	AC-20

5.7 Certificate and Verification

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-30	Certificate Issuance	Completion conditions met per BR-36/37 AND enrollment.is_active = 1	UUID v4 qr_token; certificates record; PDF generated; in-app notification. Per WF-3.5.	BR-36–39, WF-3.5, AC-11
FR-31	Certificate PDF Download	Student requests certificate download	Server validates user_id matches certificates.user_id. Returns PDF.	BR-38
FR-32	QR Code Verification	Any person visits /verify/{qr_token}	Public page: student full_name, course title, issued_at. If not found: 'Certificate not found or invalid'.	BR-39, BR-40, UC-19, AC-11

5.8 Cash-Out

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-33	Submit Cash-Out Request	User (level > 20, balance >= 100, no pending request) submits form	SELECT FOR UPDATE + conditional UPDATE (BR-47) deducts tokens; record created; ledger entry; admin notified. Per WF-3.4.	BR-30–35, BR-46, BR-47, BR-49, WF-3.4, AC-10
FR-34	Cash-Out Approval	Admin approves request	status → 'approved'; processed_at set; user notified. Admin processes external payment manually.	BR-34, WF-3.4

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-35	Cash-Out Rejection	Admin rejects (mandatory reason)	ACID transaction: tokens unconditionally restored; Refund ledger entry; status → 'rejected'; user notified.	BR-32, WF-3.4, AC-10

5.9 Search and Discovery

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-36	Course Keyword Search	User submits keyword query	Active courses matching title or description returned; paginated at 20/page.	UC-05, AC-21
FR-37	Category and Free Filter	User selects category and/or Free toggle	Active courses filtered by skill_id and/or price_tokens = 0; combinable with keyword search.	UC-05, AC-21
FR-38	Sort Options	User selects sort order	Sortable by: Newest, Price Low-High, Price High-Low, Most Enrolled.	UC-05, AC-21

5.10 Quiz Subsystem

FR-ID	Requirement	Trigger / Scenario	Expected Result	Linked
FR-39	Quiz Creation	Creator enables has_quiz = 1 and defines questions/choices/pass score	quizzes, quiz_questions, quiz_choices records saved. Min 3 questions (BR-54). Exactly 1 correct choice per question enforced by application validation AND database trigger/CHECK (BR-53).	BR-37, BR-53, BR-54, UC-14, AC-22
FR-40	Quiz Attempt and Scoring	Student with 100% completion submits quiz	quiz_attempts record created (score, passed, attempt_no, taken_at). Per WF-3.5a.	BR-37, WF-3.5a, AC-11
FR-41	Quiz Retry	Failed student clicks 'Retry Quiz'	New attempt allowed. No retry limit. All prior attempt records preserved (BR-50).	BR-37, BR-50, WF-3.5a

6. Use Cases

The following 19 use cases cover all major system interactions. Each includes actor, preconditions, trigger, main flow, alternate flows, exception flows, and postconditions.

UC-01: Register Account

Field	Detail
Actor	Guest
Preconditions	Not authenticated.
Trigger	Guest clicks 'Sign Up' and submits form.

Main Flow:

1. 1. Navigate to /register.
2. 2. Enter full_name, email, password, confirm_password.
3. 3. Server validates: all fields present; email unique; password >= 8 chars with mixed case and digit.
4. 4. System creates account (pending_verification), creates wallet (balance = 0), sends verification email.
5. 5. Guest sees: 'Check your email to activate your account'.

Alternate Flows:

- A2: Email already registered — 'Email already in use' returned without disclosing account details.

Exception Flows:

- E1: SMTP unavailable — account created, email queued for retry. User shown generic delay message.

Postcondition: Account in pending_verification state. Login blocked until email verified.

UC-02: Login

Field	Detail
Actor	Registered User
Preconditions	User has verified, active, unsuspended account.
Trigger	User submits login form.

Main Flow:

6. 1. User enters email and password.
7. 2. Server validates credentials via bcrypt.
8. 3. Session created; last_login updated; daily login reward evaluated (BR-16).

9. 4. Redirect to role-appropriate dashboard.

Alternate Flows:

- A2: Admin login — redirected to admin dashboard.

Exception Flows:

- E1: Account suspended — 'Account suspended. Contact support.' No session created.
- E2: 5 failed attempts in 10 min — account locked 30 min (FR-04).

Postcondition: Authenticated session active. last_login updated.

UC-03: Reset Password

Field	Detail
Actor	Registered User
Preconditions	User is not logged in.
Trigger	User clicks 'Forgot Password' and submits email.

Main Flow:

10. 1. User submits email.
11. 2. If found: generate UUID reset token (60-min expiry, single-use), hash stored in DB, reset link emailed.
12. 3. User clicks link; directed to /reset-password?token={token}.
13. 4. Server validates token: exists, not expired, not previously used.
14. 5. Password updated (bcrypt). Token invalidated. Redirect to login.

Exception Flows:

- E1: Email not found — 'If registered, you will receive a link.' Prevents enumeration.
- E2: Token expired — 'Link expired. Request a new one.' Token deleted.

Postcondition: Password updated. Reset token invalidated.

UC-04: Edit Profile

Field	Detail
Actor	Student or Creator
Preconditions	User is authenticated.
Trigger	User submits profile settings.

Main Flow:

15. 1. User updates bio, display name, or avatar.
16. 2. Avatar validated: jpg/png only, max 2 MB, MIME-checked server-side.
17. 3. Changes saved; success message shown.

Exception Flows:

- E1: Invalid avatar — 'Max 2 MB, jpg/png only.' No change saved.

Postcondition: Profile updated and visible on public profile page.

UC-05: Search and Browse Courses

Field	Detail
Actor	Guest, Student, Creator
Preconditions	Platform accessible.
Trigger	User enters keyword, selects category, or applies filters.

Main Flow:

18. 1. User submits keyword and/or category/Free filter.
19. 2. System queries active courses matching criteria.
20. 3. Results paginated (20/page) with sort controls.
21. 4. User selects a course to view detail (UC-06).

Alternate Flows:

- A2: No results — 'No courses found matching your criteria.'

Postcondition: Filtered, paginated, sorted course list displayed.

UC-06: View Course Detail

Field	Detail
Actor	Guest, Student, Creator
Preconditions	Course status = 'active'.
Trigger	User clicks on a course card.

Main Flow:

22. 1. Page renders: title, description, skill, creator, price, lesson list, release_threshold.
23. 2. Context-sensitive action button shown: 'Unlock Course' / 'Go to Course' / 'Edit Course'.

Exception Flows:

- E1: Course unpublished since list loaded — 'This course is not currently available.'

Postcondition: Course detail page displayed.

UC-07: Purchase a Course

Field	Detail
Actor	Student
Preconditions	Authenticated; course active; student is not creator.
Trigger	Student clicks 'Unlock Course'.

Main Flow:

24. 1. Server validates all preconditions per WF-3.1 step 3.
25. 2. SELECT FOR UPDATE + conditional UPDATE per WF-3.1 step 5.
26. 3. Enrollment (is_active=1), Payment, Escrow, Ledger records created. Student has lesson access.
27. 4. In-app notification created for student.

Alternate Flows:

- A2: Already enrolled — 'Already Owned'. No transaction.

Exception Flows:

- E1: Conditional UPDATE returns 0 rows (insufficient balance or race condition) — 'Insufficient tokens.' Full rollback.
- E2: Any transaction failure — full rollback; no partial state.

Postcondition: Enrollment, Payment, Escrow, Ledger records created. Wallet debited by conditional UPDATE.

UC-08: Watch a Lesson and Track Progress

Field	Detail
Actor	Student
Preconditions	Student enrolled; enrollment.is_active = 1.
Trigger	Student opens lesson page and starts video.

Main Flow:

28. 1. PHP proxy validates session, enrollment, and is_active = 1. Returns 403 if any fails.
29. 2. Video player posts watched_pct every 10 seconds.
30. 3. Server accumulates unique coverage per BR-41–45.

31. 4. When accumulated coverage >= 80%: server sets is_complete = 1; awards XP; evaluates daily engagement reward within SELECT FOR UPDATE transaction (BR-18); triggers streak check.
32. 5. System recalculates overall completion. If threshold reached: escrow released per WF-3.2.

Alternate Flows:

- A2: Lesson already complete — progress stored but is_complete unchanged; no duplicate XP.

Exception Flows:

- E1: Video file missing — '404: Video temporarily unavailable'.

Postcondition: Progress record updated. XP awarded. Escrow released if threshold reached.

UC-09: Take and Submit Quiz Attempt

Field	Detail
Actor	Student
Preconditions	100% video completion (all is_complete = 1). Course has has_quiz = 1. enrollment.is_active = 1.
Trigger	Student clicks 'Take Quiz'.

Main Flow:

33. 1. Server confirms 100% video completion. Rejects if any lesson incomplete.
34. 2. attempt_no = COUNT(prior attempts) + 1 for this (user_id, quiz_id).
35. 3. Questions presented (randomised if quiz.randomize_questions = 1).
36. 4. Student submits answers. Server calculates score = (correct / total) × 100.
37. 5. quiz_attempts record created (score, passed, attempt_no, taken_at).
38. 6. If passed = 1: WF-3.5 certificate issuance triggered.
39. 7. If passed = 0: score shown; 'Retry when ready.'

Exception Flows:

- E1: Attempt before 100% completion — 'Complete all lessons first.' Rejected.
- E2: enrollment.is_active = 0 — access denied; quiz blocked.

Postcondition: quiz_attempts record created. Certificate issued if passed = 1.

UC-10: Creator Edit Active Course

Field	Detail
Actor	Creator
Preconditions	Creator authenticated. Course status = 'active'. Creator owns the course.
Trigger	Creator clicks 'Edit Course' and submits changes.

Main Flow:

40. 1. Creator modifies course fields or lessons.
41. 2. System saves changes; course status → 'pending_review'.
42. 3. Enrolled students retain access; new enrollments disabled. Admin notified.

Exception Flows:

- E1: Creator edits course they do not own — 403 Forbidden.

Postcondition: Course in pending_review. Existing enrollments and access unaffected.

UC-11: Admin Unpublish Active Course

Field	Detail
Actor	Admin
Preconditions	Admin authenticated. Course status = 'active'.
Trigger	Admin clicks 'Unpublish' and enters mandatory reason.

Main Flow:

43. 1. Admin enters mandatory reason.
44. 2. course.status → 'rejected'; in-app notification sent to creator.
45. 3. New enrollments blocked; existing enrolled students retain full access (is_active not changed).

Exception Flows:

- E1: Admin submits without reason — form blocked.

Postcondition: Course status = 'rejected'. Creator notified. Existing students unaffected.

UC-12: Receive Certificate

Field	Detail
Actor	System (auto-triggered)
Preconditions	Completion conditions met per BR-36/37. enrollment.is_active = 1.
Trigger	End of WF-3.2 step 8 or WF-3.5a step 5.

Main Flow:

46. 1. Check UNIQUE(user_id, course_id) in certificates — abort if exists.
47. 2. Generate UUID v4 qr_token. Create certificates record.
48. 3. Generate PDF (student full_name, course title, issued_at, QR code linking to /verify/{qr_token}).
49. 4. Store PDF; update certificates.pdf_path. Create in-app notification for student.

Exception Flows:

- E1: PDF generation fails — certificate record still created; PDF generation retried. Student notified of delay.

Postcondition: Certificate record exists. PDF stored. QR verification active.

UC-13: Raise a Dispute

Field	Detail
Actor	Student
Preconditions	Enrolled; escrow.status = 'held'; no prior dispute; within 30 days of enrolled_at.
Trigger	Student clicks 'Raise Dispute' and submits reason.

Main Flow:

50. 1. Server validates eligibility per BR-24, BR-25, BR-26.
51. 2. SELECT FOR UPDATE on escrow row (BR-46). ACID transaction: escrow.status → 'frozen'; payment.status → 'disputed' (BR-48); Dispute record created (pending).
52. 3. In-app notifications created for admin, student, and creator.

Exception Flows:

- E1: Escrow already released — 'Dispute cannot be raised after funds have been released.'
- E2: Duplicate — 'You have already filed a dispute for this course.'
- E3: Window expired — 'The 30-day dispute window has closed.'

Postcondition: Escrow frozen. Payment disputed. Dispute record (pending). Both parties notified.

UC-14: Create and Submit a Course

Field	Detail
Actor	Creator
Preconditions	User has creator role. Account active.
Trigger	Creator fills course creation form and clicks 'Submit for Review'.

Main Flow:

53. 1. Creator enters title, description, skill_id, price_tokens, release_threshold (>= 20), optional quiz config (min 3 questions, exactly 1 correct choice per question — BR-53, BR-54).
54. 2. Creator uploads at least 1 lesson.
55. 3. Server validates all fields and video file. Course status → 'pending_review'. Admin notified.

Alternate Flows:

- A2: Save as draft — status = 'draft'; not visible to admin or students.

Exception Flows:

- E1: Video MIME-type check fails — 'Invalid file type.' Course not submitted.
- E2: Quiz has fewer than 3 questions — 'Quiz requires at least 3 questions.' Submission blocked.

Postcondition: Course in pending_review. Admin queue updated.

UC-15: Admin Approve / Reject Course

Field	Detail
Actor	Admin
Preconditions	Admin authenticated. Course with status = 'pending_review' exists.
Trigger	Admin opens course in review queue.

Main Flow:

56. 1. Admin reviews course content, lessons, pricing.
57. 2. Approve → status = 'active'; creator notified.
58. 3. OR Reject (mandatory reason) → status = 'rejected'; creator notified with reason.

Exception Flows:

- E1: Rejection without reason — form blocked.

Postcondition: Course status updated. Creator notified.

UC-16: Admin Resolve a Dispute

Field	Detail
Actor	Admin
Preconditions	Dispute resolution = 'pending'. Escrow status = 'frozen'.
Trigger	Admin opens dispute detail in dashboard.

Main Flow:

59. 1. Admin reviews dispute reason, course content, student progress evidence.
60. 2. Admin selects resolution per WF-3.3 step 7.
61. 3. SELECT FOR UPDATE on escrow and wallet rows (BR-46). ACID transaction: escrow, payment, enrollment.is_active, and wallet updated atomically (BR-48).
62. 4. In-app notifications for both parties. dispute.resolved_at and admin_id recorded.

Exception Flows:

- E1: Escrow no longer 'frozen' — system alerts admin. Action blocked.

Postcondition: Dispute resolved. Escrow and payment status updated atomically. Parties notified.

UC-17: Request Cash-Out

Field	Detail
Actor	Student or Creator
Preconditions	User level > 20. wallet.token_balance >= 100. No pending request.
Trigger	User submits cash-out form.

Main Flow:

63. 1. Server validates per WF-3.4 step 2.
64. 2. User selects method and enters masked account identifier only.
65. 3. Preview shown. User confirms.
66. 4. SELECT FOR UPDATE + conditional UPDATE per WF-3.4 step 5. If 0 rows affected: rejected.
67. 5. cash_out_requests record (pending); Cash_Out ledger entry. Admin notified.

Exception Flows:

- E1: Any validation failure — specific error; no tokens deducted.

Postcondition: Tokens deducted via conditional UPDATE. Cash-out record pending admin action.

UC-18: Admin Approve or Reject Cash-Out

Field	Detail
Actor	Admin
Preconditions	Admin authenticated. cash_out_requests record with status = 'pending' exists.
Trigger	Admin opens cash-out request in dashboard.

Main Flow:

68. 1. Admin reviews user level, token amount, net payout, method.

69. 2. Approve → status = 'approved'; processed_at set; user notified. Admin processes external payment manually.
70. 3. OR Reject (mandatory reason) → ACID transaction: unconditionally restore tokens; Refund ledger entry; status = 'rejected'; user notified with reason.

Exception Flows:

- E1: Rejection without reason — form blocked.

Postcondition: Request status updated. If rejected: tokens restored to wallet. User notified.

UC-19: Verify Certificate via QR

Field	Detail
Actor	Any person (no login required)
Preconditions	Person has a certificate QR code.
Trigger	Person scans QR or visits /verify/{qr_token}.

Main Flow:

71. 1. Server receives qr_token from URL.
72. 2. Queries certificates table for matching qr_token.
73. 3. If found: display student full_name, course title, issued_at, 'Verified' status.
74. 4. If not found: 'Certificate not found or invalid'.

Exception Flows:

- E1: DB temporarily unavailable — 'Verification service temporarily unavailable.'

Postcondition: Verification result displayed. No personal data beyond name, course, and date shown.

7. Database Schema

All tables use InnoDB engine (required for row-level locking and ACID transactions), UTF8MB4 charset. All names use snake_case consistently. All foreign keys define explicit referential actions.

Canonical table names
users, wallet, categories, skills, courses, lessons, enrollments, progress, payments, escrow, disputes, cash_out_requests, quizzes, quiz_questions, quiz_choices, quiz_attempts, notifications, token_ledger, certificates, achievements, user_achievements

7.1 users

Field	Type	Constraints	Notes
user_id	INT	PK, AUTO_INCREMENT	
full_name	VARCHAR(150)	NOT NULL	
email	VARCHAR(255)	UNIQUE, NOT NULL	
password_hash	VARCHAR(255)	NOT NULL	bcrypt, cost factor >= 12
roles	SET('student','creator','admin')	DEFAULT 'student'	Prototype simplicity choice — see design note in §3.2
is_verified	TINYINT(1)	DEFAULT 0	1 = email verified; login blocked until 1
is_suspended	TINYINT(1)	DEFAULT 0	1 = suspended by admin
level	INT	DEFAULT 1, CHECK >= 1	Must be > 20 for cash-out
xp	INT	DEFAULT 0, CHECK >= 0	Current XP within current level
streak_days	INT	DEFAULT 0, CHECK >= 0	Consecutive active day counter
last_streak_date	DATE	NULLABLE	Last day a qualifying lesson completion occurred
last_login	DATETIME	NULLABLE	Updated on every successful login
failed_login_count	INT	DEFAULT 0	Resets to 0 on successful login
lockout_until	DATETIME	NULLABLE	Account locked if value is in the future
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

7.2 wallet

Field	Type	Constraints	Notes
wallet_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE CASCADE, UNIQUE, NOT NULL	One wallet per user. Single authoritative balance store.
token_balance	INT	DEFAULT 0, CHECK >= 0	Deductions use conditional UPDATE (BR-47). SELECT FOR UPDATE applied on every financial operation (BR-46).
lifetime_earned	INT	DEFAULT 0	Total tokens ever credited. Used for achievement 'tokens_earned' condition.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Critical rule
The users table contains NO token balance field. wallet.token_balance is the sole source of truth. All deductions use the conditional UPDATE pattern (BR-47) with SELECT FOR UPDATE (BR-46) to prevent race conditions.

7.3 categories

Field	Type	Constraints	Notes
category_id	INT	PK, AUTO_INCREMENT	
name	VARCHAR(150)	NOT NULL, UNIQUE	e.g., Technical, Creative, Business, Language

7.4 skills

Field	Type	Constraints	Notes
skill_id	INT	PK, AUTO_INCREMENT	
category_id	INT	FK → categories(category_id) ON DELETE RESTRICT, NOT NULL	Normalised reference
skill_name	VARCHAR(150)	NOT NULL	e.g., JavaScript, Graphic Design

7.5 courses

Field	Type	Constraints	Notes
course_id	INT	PK, AUTO_INCREMENT	
creator_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	
skill_id	INT	FK → skills(skill_id) ON DELETE RESTRICT, NOT NULL	
title	VARCHAR(255)	NOT NULL	
description	TEXT	NULLABLE	
price_tokens	INT	NOT NULL, DEFAULT 0, CHECK >= 0	0 = free course
release_threshold	INT	NOT NULL, DEFAULT 20, CHECK >= 20	Enforced at DB and application levels
has_quiz	TINYINT(1)	DEFAULT 0	1 = quiz required for certificate
quiz_pass_score	INT	NULLABLE, CHECK BETWEEN 1 AND 100	Valid only when has_quiz = 1
status	ENUM('draft','pending_review','active','rejected')	DEFAULT 'pending_review'	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

7.6 lessons

Field	Type	Constraints	Notes
lesson_id	INT	PK, AUTO_INCREMENT	
course_id	INT	FK → courses(course_id) ON DELETE CASCADE, NOT NULL	

Field	Type	Constraints	Notes
title	VARCHAR(255)	NOT NULL	
description	TEXT	NULLABLE	
video_path	VARCHAR(500)	NOT NULL	UUID-based filename; stored outside public web root
duration_seconds	INT	NOT NULL, CHECK > 0	Used for server-side coverage % calculation
order_index	INT	NOT NULL	UNIQUE(course_id, order_index)
is_free_preview	TINYINT(1)	DEFAULT 0	1 = accessible without enrollment
status	ENUM('draft','published')	DEFAULT 'draft'	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

7.7 enrollments

Field	Type	Constraints	Notes
enrollment_id	INT	PK, AUTO_INCREMENT	
student_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	
course_id	INT	FK → courses(course_id) ON DELETE RESTRICT, NOT NULL	
is_active	TINYINT(1)	DEFAULT 1	0 = access revoked (student-win dispute per BR-27). All lesson streaming, progress, and certificate issuance require is_active = 1.
enrolled_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Used for 30-day dispute window (BR-26)
UNIQUE	(student_id, course_id)	DB-level constraint	Prevents duplicate enrollment under concurrency. Combined with SELECT FOR UPDATE in WF-3.1 (BR-49).

7.8 progress

Field	Type	Constraints	Notes
progress_id	INT	PK, AUTO_INCREMENT	
student_id	INT	FK → users(user_id) ON DELETE CASCADE, NOT NULL	
lesson_id	INT	FK → lessons(lesson_id) ON DELETE CASCADE, NOT NULL	
watched_pct	INT	DEFAULT 0, CHECK BETWEEN 0 AND 100	Server-accumulated unique coverage; not raw client value
is_complete	TINYINT(1)	DEFAULT 0	Set to 1 only by server when accumulated coverage >= 80% (BR-42). Never set by client.
completed_at	TIMESTAMP	NULLABLE	
UNIQUE	(student_id, lesson_id)	DB-level constraint	One progress record per student per lesson

7.9 payments

Field	Type	Constraints	Notes
payment_id	INT	PK, AUTO_INCREMENT	
student_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	
course_id	INT	FK → courses(course_id) ON DELETE RESTRICT, NOT NULL	
amount_tokens	INT	NOT NULL, CHECK > 0	
status	ENUM('pending','released','disputed','cancelled')	DEFAULT 'pending'	Must always mirror escrow.status per the mapping: held→pending, released→released, frozen→disputed,

Field	Type	Constraints	Notes
			cancelled→cancelled. Enforced by BR-48.
paid_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
UNIQUE	(student_id, course_id)	DB-level constraint	Idempotency against concurrent purchases (BR-49)

Escrow ↔ Payment synchronisation
Escrow is the authoritative source of truth for payment lifecycle state. payment.status must always be updated to mirror escrow.status transitions within the same ACID transaction. No code path may update escrow.status without simultaneously updating payment.status. This is enforced as a mandatory invariant per BR-48.

7.10 escrow

Field	Type	Constraints	Notes
escrow_id	INT	PK, AUTO_INCREMENT	
payment_id	INT	FK → payments(payment_id) ON DELETE RESTRICT, UNIQUE, NOT NULL	One escrow per payment
creator_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	
amount_tokens	INT	NOT NULL, CHECK > 0	
status	ENUM('held','released','frozen','cancelled')	DEFAULT 'held'	Authoritative lifecycle state. Status changes trigger corresponding payment.status update (BR-48). SELECT FOR UPDATE required before any change (BR-46).
frozen_reason	TEXT	NULLABLE	Admin note on freeze reason

Field	Type	Constraints	Notes
released_at	TIMESTAMP	NULLABLE	

7.11 disputes

Field	Type	Constraints	Notes
dispute_id	INT	PK, AUTO_INCREMENT	
escrow_id	INT	FK → escrow(escrow_id) ON DELETE RESTRICT, UNIQUE, NOT NULL	UNIQUE enforces one dispute per escrow/enrollment (BR-24). ON DELETE RESTRICT preserves audit trail (BR-50).
raised_by	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	Must be student; validated application-side
reason	TEXT	NOT NULL	Min 20 characters enforced application-side
admin_id	INT	FK → users(user_id) NULLABLE	Set when admin assigned; ON UPDATE CASCADE
resolution	ENUM('pending','resolved_creator','resolved_student','cancelled')	DEFAULT 'pending'	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
resolved_at	TIMESTAMP	NULLABLE	Set when resolution != 'pending'

7.12 cash_out_requests

Field	Type	Constraints	Notes
cashout_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	ON DELETE RESTRICT — preserves audit trail (BR-50)
amount_tokens	INT	NOT NULL, CHECK >= 100	
usd_equivalent	DECIMAL(10,2)	NOT NULL	amount_tokens × 0.10
platform_commission	DECIMAL(10,2)	NOT NULL	usd_equivalent × 0.07
net_payout	DECIMAL(10,2)	NOT NULL	usd_equivalent − platform_commission
method	ENUM('visa','apple_pay','paypal')	NOT NULL	
account_identifier	VARCHAR(255)	NULLABLE	ONLY masked identifiers stored (e.g., last 4 digits, partial email). Full card numbers, bank details, or full PayPal IDs must NEVER be stored (BR-50).
status	ENUM('pending','approved','rejected')	DEFAULT 'pending'	
rejection_reason	TEXT	NULLABLE	Mandatory when admin rejects
requested_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
processed_at	TIMESTAMP	NULLABLE	

7.13 quizzes

Field	Type	Constraints	Notes
quiz_id	INT	PK, AUTO_INCREMENT	
course_id	INT	FK → courses(course_id) ON DELETE CASCADE, UNIQUE, NOT NULL	One quiz per course
time_limit_minutes	INT	NULLABLE	NULL = no time limit

Field	Type	Constraints	Notes
randomize_questions	TINYINT(1)	DEFAULT 0	1 = randomise question order per attempt

7.14 quiz_questions

Field	Type	Constraints	Notes
question_id	INT	PK, AUTO_INCREMENT	
quiz_id	INT	FK → quizzes(quiz_id) ON DELETE CASCADE, NOT NULL	
question_text	TEXT	NOT NULL	
order_index	INT	NOT NULL	UNIQUE(quiz_id, order_index)

7.15 quiz_choices

Field	Type	Constraints	Notes
choice_id	INT	PK, AUTO_INCREMENT	
question_id	INT	FK → quiz_questions(question_id) ON DELETE CASCADE, NOT NULL	References quiz_questions — not a generic 'questions' table
choice_text	VARCHAR(500)	NOT NULL	
is_correct	TINYINT(1)	DEFAULT 0	Exactly one is_correct = 1 per question_id. Enforced by application validation (BR-53) and a MySQL AFTER INSERT/UPDATE trigger that rejects any insert/update that would leave a question with 0 or > 1 correct choices.

Quiz correctness enforcement
A MySQL BEFORE INSERT / BEFORE UPDATE trigger on quiz_choices raises an error if the insert/update would result in a question_id having a count of correct choices that is not exactly 1 after the operation. Application-side validation (BR-53) serves as the first line of defence; the trigger is the database-level guarantee.

7.16 quiz_attempts

Field	Type	Constraints	Notes
attempt_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	ON DELETE RESTRICT — preserves attempt history (BR-50)
quiz_id	INT	FK → quizzes(quiz_id) ON DELETE RESTRICT, NOT NULL	
attempt_no	INT	NOT NULL	Sequential attempt number per (user_id, quiz_id). Calculated as COUNT(prior attempts) + 1 on insert.
score	DECIMAL(5,2)	NOT NULL	Percentage 0.00 – 100.00
passed	TINYINT(1)	NOT NULL	1 if score >= courses.quiz_pass_score at time of attempt
taken_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

7.17 notifications

Field	Type	Constraints	Notes
notification_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE CASCADE, NOT NULL	
type	ENUM('course_approved','course_rejected','dispute_filed','dispute_resolved','cashout_approved','cashout_rejected','certificate_issued','achievement_unlocked','escrow_released','streak_bonus','general')	NOT NULL	
title	VARCHAR(255)	NOT NULL	
body	TEXT	NOT NULL	
is_read	TINYINT(1)	DEFAULT 0	

Field	Type	Constraints	Notes
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

Notification delivery model
In-app notifications are persisted in this table and are the primary reliable notification channel. Email notifications are best-effort: sent via SMTP but delivery success is not tracked and failed emails are not retried or logged in the database in this prototype.

7.18 token_ledger

Field	Type	Constraints	Notes
log_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	ON DELETE RESTRICT: preserves immutable audit trail even if user deleted (BR-50, BR-51)
action_type	ENUM('Daily_Login','Daily_Engagement','Streak_Bonus','Purchase','Escrow_Release','Cash_Out','Refund','Achievement_Reward','Registration_Bonus')	NOT NULL	
amount	INT	NOT NULL	Positive = credit; negative = debit
balance_after	INT	NOT NULL	wallet.token_balance after this transaction

Field	Type	Constraints	Notes
reference_type	ENUM('payment','escrow','dispute','cashout','achievement','none')	DEFAULT 'none'	Identifies what reference_id points to
reference_id	INT	NULLABLE	ID of the related record; interpreted with reference_type
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Immutable — never updated after insert (BR-50)

7.19 certificates

Field	Type	Constraints	Notes
cert_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE RESTRICT, NOT NULL	ON DELETE RESTRICT — preserves certificate records (BR-50)
course_id	INT	FK → courses(course_id) ON DELETE RESTRICT, NOT NULL	
qr_token	VARCHAR(64)	UNIQUE, NOT NULL	UUID v4 format; never reused
pdf_path	VARCHAR(500)	NULLABLE	NULL if generation pending retry
issued_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
UNIQUE	(user_id, course_id)	DB-level constraint	One certificate per student per course (BR-38)

7.20 achievements and 7.21 user_achievements

Field	Type	Constraints	Notes
achievement_id	INT	PK, AUTO_INCREMENT	
name	VARCHAR(150)	NOT NULL, UNIQUE	
description	TEXT	NOT NULL	
icon_path	VARCHAR(255)	NULLABLE	
condition_type	ENUM('courses_completed','streak_days','tokens_earned','level_reached','courses_created')	NOT NULL	Scoped queries per BR-52 use this field to avoid full-table scans
condition_value	INT	NOT NULL, CHECK > 0	
token_reward	INT	DEFAULT 0, CHECK >= 0	0 = no reward; > 0 requires SELECT FOR UPDATE on wallet (BR-46)
is_active	TINYINT(1)	DEFAULT 1	0 = deactivated; no longer triggers (BR-52)

Field (user_achievements)	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK → users(user_id) ON DELETE CASCADE, NOT NULL	
achievement_id	INT	FK → achievements(achievement_id) ON DELETE RESTRICT, NOT NULL	ON DELETE RESTRICT — preserves earned history (BR-50)
earned_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

Field (user_achievements)	Type	Constraints	Notes
UNIQUE	(user_id, achievement_id)	DB-level constraint	Cannot earn same achievement twice

8. Non-Functional Requirements

NFR-ID	Category	Requirement	Test Scenario	Acceptance Criteria
NFR-01	Security	Password hashing	Register new account	Password stored as bcrypt hash, cost factor >= 12. Plaintext never stored or logged.
NFR-02	Security	HTTPS enforcement	Attempt HTTP access	All HTTP redirected to HTTPS (301). HSTS header present.
NFR-03	Security	CSRF protection	Submit state-changing form with missing CSRF token	Request rejected with 403.
NFR-04	Security	XSS prevention	Submit <script> tag in course description or bio	Content HTML-escaped on output. CSP header set.
NFR-05	Security	SQL injection prevention	Submit SQL payload in any text input	All queries use prepared statements. No raw concatenation in SQL.
NFR-06	Security	File upload restriction	Upload .php disguised as .mp4	MIME-checked server-side; UUID-renamed; stored outside web root.
NFR-07	Security	Session security	Inspect session cookie	HttpOnly + Secure flags set. Session regenerated on login. Timeout 60 min inactivity.
NFR-08	Security	Admin route protection	Direct URL access to /admin/* without admin role	403 returned. All admin routes check role server-side.
NFR-09	Integrity	Transactional atomicity	Disconnect during token deduction in purchase	Full rollback. No partial state. Wallet balance unchanged.
NFR-10	Integrity	Audit trail completeness and immutability	Inspect token_ledger after any token event	Every token movement has a ledger row with reference_type and reference_id. Ledger rows are never deleted or updated (BR-50).
NFR-11	Security	Login throttling and lockout	Incorrect password 5 times in 10 minutes	Account locked 30 minutes. lockout_until set. Resets on successful login.
NFR-12	Performance	Search page load	100 concurrent users search simultaneously	Page load < 2 seconds under simulated load.
NFR-13	Performance	Video proxy response	Single user requests a lesson video	First-byte response < 1 second on local server. No direct URL accessible.

NFR-ID	Category	Requirement	Test Scenario	Acceptance Criteria
NFR-14	Performance	Transactional responsiveness	Single purchase or cash-out submission under normal load	System processes full ACID transactions (including SELECT FOR UPDATE) within 1 second under normal single-server load. This is a target, not a hard guarantee — acceptable degradation under peak load should be documented.
NFR-15	Usability	Responsive design	Open platform on 375px-wide mobile	All pages render without horizontal scrolling. Touch targets >= 44 x 44 px.
NFR-16	Usability	Error message clarity	Trigger any validation error	Error messages identify the exact field or rule that failed.
NFR-17	Reliability	Data backup and recovery	Simulate accidental table deletion	System restores from most recent 24-hour backup within 4 hours.
NFR-18	Availability	Uptime	Monitor server over 30 days	Uptime >= 99.5%.
NFR-19	Maintainability	3-tier architecture	Code review	No inline SQL in HTML files. Presentation, logic, and data layers clearly separated.
NFR-20	Accessibility	Keyboard navigation	Navigate site using keyboard only	All interactive elements reachable via Tab key. Focus indicator always visible.
NFR-21	Privacy	Public profile data	Inspect public student profile	Only full_name, bio, avatar, level, and earned achievements are publicly visible. Email, wallet, and transaction history never exposed.
NFR-22	Compatibility	Browser compatibility	Open platform in each listed browser	Platform renders correctly and all features function in Chrome 110+, Firefox 110+, Edge 110+, Safari 16+ on desktop and mobile.
NFR-23	Accessibility	Video learning accessibility	Review lesson video pages	Each lesson page provides a link to a text transcript or description. Best-effort for this prototype.
NFR-24	Security	Endpoint authorization	Access any protected endpoint	Every protected route verifies user role at the controller layer before executing any business logic. Failure to pass role check returns 403 immediately. This applies to all endpoints not designated as public.
NFR-25	Performance	Event-driven achievement evaluation	Trigger achievement check	Achievement evaluation queries are scoped to the relevant condition_type (BR-52). Full-

NFR-ID	Category	Requirement	Test Scenario	Acceptance Criteria
			after any qualifying event	table scans of all achievements are not permitted per event trigger. Verified by code review.

9. External Interface Requirements

9.1 Required UI Screens

Screen	Actor(s)	Key Required Elements
Homepage / Course Catalogue	Guest, Student, Creator	Search bar, category filter, Free toggle, sort dropdown, course cards (title, creator, price, enrollment count), pagination
Course Detail Page	All	Title, description, skill, creator, price, lesson list, release_threshold, context-sensitive action button (Unlock / Go-to-Course / Edit)
Course Learning Page	Student (is_active=1)	Video player, lesson navigation sidebar, overall progress bar, Raise Dispute button, Download Certificate button when earned. Revoked enrollments (is_active=0) see access-denied message.
Creator Dashboard	Creator	Course list with status badges, earnings summary (released/pending escrow), total students per course
Course Creation / Edit Form	Creator	All course fields, ordered lesson upload, quiz configuration section
Student Dashboard / Profile	Student	Enrolled courses with progress (is_active=1 only in Active Learning view), achievements grid, certificates list, token balance, level/XP bar
Admin Dashboard — Overview	Admin	Total users, active courses, open disputes, pending cash-outs, token volume, commission earned
Admin — Course Review Queue	Admin	Pending courses; Approve and Reject (mandatory reason) buttons
Admin — Dispute Management	Admin	Active dispute list; dispute detail; Resolve-Creator / Resolve-Student buttons
Admin — Cash-Out Queue	Admin	Pending requests list; Approve and Reject (mandatory reason) buttons
Cash-Out Request Form	Student, Creator	Token amount input, live payout preview, payment method selector. Masked account identifier input only.
Certificate Verification Page	Public (no login)	QR scan result: student name, course title, issue date, verification status.
Notification Centre	All authenticated	Notification list with type icon, title, timestamp, read/unread; Mark All Read button

9.2 Software Interface Specifications

Interface	Type	Purpose	Requirement
SMTP Email Service	External API	Registration verification, password reset, notifications (best-effort)	PHPMailer + SMTP. Credentials in server-side environment config only. Delivery failures logged server-side; not surfaced to users.

Interface	Type	Purpose	Requirement
Dompdf (or equivalent)	PHP Library	Generate certificate PDF	Installed via Composer. Certificate DB record created first; PDF retried on failure.
PHP QR Code Library	PHP Library	Generate QR code embedded in certificate	PNG output embedded in Dompdf HTML template.
MySQL 8.0+ (InnoDB)	Database	All persistent data. Row-level locking (SELECT FOR UPDATE) required for all financial operations.	InnoDB engine mandatory. UTF8MB4 charset. Strict mode. Binary log enabled. Transaction isolation level: REPEATABLE READ (default) with explicit SELECT FOR UPDATE for financial rows.

10. Risk Analysis and Mitigation

RK-ID	Category	Risk Scenario	Mitigation	Expected Outcome
RK-01	Logic	User manipulates course price via browser DevTools	Server-side price fetched from DB; client-supplied price ignored	Correct price always charged
RK-02	Concurrency	Double-click 'Unlock' — race condition	UNIQUE(student_id, course_id) + SELECT FOR UPDATE + conditional UPDATE (BR-46, BR-47, BR-49)	Only 1 enrollment; only 1 deduction
RK-03	Security	Attacker uploads PHP shell disguised as video	MIME-type check server-side; UUID rename; stored outside web root	File rejected or rendered harmless
RK-04	Integrity	Network drops after wallet deduction but before enrollment	Full ACID transaction with rollback on any failure	Tokens restored; no orphan records
RK-05	Privacy	Attacker guesses video URL of paid lesson	PHP proxy validates session + enrollment.is_active = 1	403 without valid active enrollment
RK-06	Security	Session hijacking via cookie theft	HttpOnly + Secure flags; HTTPS; 60-min session timeout	Stolen cookie unusable
RK-07	Fraud	Creator sets release_threshold to 1%	DB CHECK >= 20 + server validation	Values below 20% rejected at both layers
RK-08	Dispute abuse	Student files multiple disputes to freeze funds	UNIQUE(escrow_id) in disputes; max 1 per enrollment (BR-24)	Only one dispute per enrollment
RK-09	Data integrity	Dual wallet balance divergence	users table has no token balance field; wallet is sole source (BR-04)	Single source of truth; no drift
RK-10	Financial	Admin rejects cash-out; tokens already deducted	Token restoration in same ACID transaction as rejection (BR-32)	User never loses tokens from admin rejection
RK-11	Progress fraud	Student sends fabricated watched_pct values	BR-41–45: server accumulates unique coverage; large jumps discarded	Fabricated completion not possible without genuine playback

RK-ID	Category	Risk Scenario	Mitigation	Expected Outcome
RK-12	Financial	Full payment credentials stored in database	Only masked identifiers stored (BR-50); full credentials never accepted	Sensitive financial data never at risk
RK-13	Concurrency	Double escrow release from concurrent progress reports	SELECT FOR UPDATE on escrow row before release attempt (BR-46, BR-23)	Only one release; second attempt sees status != 'held' and aborts
RK-14	Concurrency	Double daily reward granted under concurrent logins	SELECT FOR UPDATE on wallet row; daily check within same transaction (BR-18, BR-46)	Only one daily reward credited per user per day
RK-15	Data integrity	Payment and escrow status divergence	BR-48: payment.status updated in same transaction as every escrow.status change	payment.status always mirrors escrow.status; no divergence possible

11. Acceptance Criteria

Each criterion defines a testable pass/fail condition. All marked criteria must pass before the system is considered ready for submission.

AC-ID	Feature	Pass Condition
AC-01	Registration	Account created; verification email sent; login blocked until verified; 100-token bonus credited on activation.
AC-02	Login throttle	Account locked after 5 failed attempts in 10 min; lockout_until set; resets on successful login.
AC-03	Course purchase	Conditional UPDATE verifies 1 row affected; ACID transaction: enrollment (is_active=1) + payment (pending) + escrow (held) + ledger created; student has lesson access.
AC-04	Duplicate purchase	Second purchase: 'Already enrolled'; zero deduction; DB record counts unchanged.
AC-05	Video access control	Request to PHP proxy without active enrollment (is_active=1): 403. Direct URL never accessible.
AC-06	Progress tracking	is_complete set to 1 only by server when server-accumulated unique coverage >= 80%. Client cannot override. Large jumps in watched_pct discarded.
AC-07	Escrow release	Escrow → 'released' and payment → 'released' and creator wallet credited in same ACID transaction at threshold. Never triggers if frozen. SELECT FOR UPDATE prevents double release.
AC-08	Streak bonus	+100 tokens credited when streak_days mod 3 == 0 after daily lesson completion. Resets to 1 if gap > 1 calendar day.
AC-09	Dispute eligibility	Second dispute: rejected. After release: rejected. After 30 days: rejected. Each with specific error. payment.status → 'disputed' in same transaction as escrow → 'frozen'.
AC-10	Cash-out rejection	On rejection: tokens restored AND Refund ledger entry in same ACID transaction. Status = 'rejected'.
AC-11	Certificate	Issued only when all is_complete=1 AND enrollment.is_active=1 (AND quiz passed=1 if has_quiz=1). UUID qr_token. Verification page public. Revoked enrollment never produces certificate.
AC-12	File upload security	PHP file: rejected. Stored file UUID-named and inaccessible via direct URL.
AC-13	CSRF	Missing/wrong CSRF token returns 403.
AC-14	XSS	Script tag rendered as escaped text; not executed.
AC-15	Concurrent purchase	10 simultaneous purchase requests by same user for same course: exactly 1 enrollment and 1 wallet deduction.
AC-16	Concurrency — double escrow release	2 simultaneous progress reports that both cross the release_threshold result in exactly 1 escrow release and 1 creator wallet credit.

AC-ID	Feature	Pass Condition
AC-17	Conditional UPDATE integrity	Purchase with wallet balance = course price in concurrent scenario: exactly 1 transaction succeeds; all others see 0 rows affected and roll back.
AC-18	Escrow-payment sync	At any point in time: payment.status always mirrors escrow.status per the defined mapping. No diverged states exist in the database.
AC-19	Authorization enforcement	A request to any protected endpoint (/admin/*, /course/purchase, etc.) without the required role returns 403 before any business logic executes.
AC-20	Achievement management	Admin creates or deactivates achievement; deactivated no longer unlocks; existing earned records unaffected.
AC-21	Search and filter	Keyword search returns only active courses; category/Free filter combinable; paginated 20/page.
AC-22	Quiz creation	Creator creates quiz with >= 3 questions; each question has exactly 1 correct choice; database trigger prevents inserting a second correct choice for the same question.

12. Traceability Matrix

Maps each FR to its linked business rules, primary database tables, workflow reference, and acceptance criterion. All table names use canonical snake_case from Section 7.

FR-ID	Feature	Business Rules	Primary Tables	Workflow	AC-ID
FR-01	User Registration	BR-02	users, wallet	—	AC-01
FR-02	Email Verification	BR-02	users, wallet, token_ledger	—	AC-01
FR-03	User Login	BR-16	users, wallet, token_ledger	—	AC-02
FR-04	Login Throttle	—	users	—	AC-02
FR-05	Password Reset	—	users	—	—
FR-06	Profile Management	—	users	—	—
FR-07	Account Suspension	—	users	—	—
FR-08	Course Creation	BR-20	courses, lessons	—	—
FR-09	Submission Validation	BR-20	courses	—	—
FR-10	Admin Course Approval	—	courses, notifications	—	—
FR-11	Course Re-submission	—	courses	—	—
FR-12	Edit Active Course	—	courses	—	—
FR-13	Admin Unpublish	—	courses, notifications	—	—
FR-14	Video Upload	—	lessons	—	AC-12
FR-15	Course Purchase	BR-04, BR-19, BR-46, BR-47, BR-49	wallet, enrollments, payments, escrow, token_ledger	WF-3.1	AC-03, AC-04, AC-15, AC-17
FR-16	Duplicate Purchase Prevention	BR-04, BR-49	enrollments	WF-3.1	AC-04
FR-17	Self-Purchase Prevention	—	enrollments	—	—

FR-ID	Feature	Business Rules	Primary Tables	Workflow	AC-ID
FR-18	Insufficient Balance	BR-04, BR-47	wallet	—	AC-17
FR-19	Video Access Control	BR-27	enrollments	WF-3.2	AC-05
FR-20	Progress Tracking	BR-41–45, BR-46	progress, wallet, token_ledger	WF-3.2	AC-06
FR-21	Escrow Release	BR-21, BR-23, BR-46, BR-48	escrow, wallet, payments, token_ledger	WF-3.2	AC-07, AC-16, AC-18
FR-22	Course Completion	BR-06, BR-36, BR-37	progress, users, enrollments	WF-3.2, WF-3.5	AC-11
FR-23	Raise Dispute	BR-24–26, BR-46, BR-48	disputes, escrow, payments, notifications	WF-3.3	AC-09, AC-18
FR-24	Dispute Eligibility Check	BR-24, BR-25, BR-26	disputes, escrow, enrollments	WF-3.3	AC-09
FR-25	Admin Dispute Resolution	BR-27–29, BR-46, BR-48	disputes, escrow, wallet, token_ledger, enrollments, payments, notifications	WF-3.3	AC-11, AC-18
FR-26	Streak Bonus	BR-11–15, BR-46	users, wallet, token_ledger, notifications	WF-3.6	AC-08
FR-27	XP and Level-Up	BR-06–10	users, notifications	—	—
FR-28	Achievement Unlock	BR-46, BR-52	achievements, user_achievements, wallet, token_ledger, notifications	WF-3.7	AC-20
FR-29	Admin Achievement Mgmt	—	achievements	—	AC-20
FR-30	Certificate Issuance	BR-36–39	certificates, quiz_attempts, enrollments, notifications	WF-3.5	AC-11
FR-31	Certificate Download	BR-38	certificates	—	AC-11
FR-32	QR Verification	BR-39, BR-40	certificates	—	AC-11
FR-33	Submit Cash-Out	BR-30–35, BR-46, BR-47, BR-49	cash_out_requests, wallet, token_ledger, notifications	WF-3.4	AC-10, AC-17
FR-34	Cash-Out Approval	BR-34	cash_out_requests, notifications	WF-3.4	—

FR-ID	Feature	Business Rules	Primary Tables	Workflow	AC-ID
FR-35	Cash-Out Rejection	BR-32	cash_out_requests, wallet, token_ledger, notifications	WF-3.4	AC-10
FR-36	Course Search	—	courses	—	AC-21
FR-37	Category / Free Filter	—	courses, skills	—	AC-21
FR-38	Sort Options	—	courses, enrollments	—	AC-21
FR-39	Quiz Creation	BR-37, BR-53, BR-54	quizzes, quiz_questions, quiz_choices	—	AC-22
FR-40	Quiz Attempt and Scoring	BR-37	quiz_attempts	WF-3.5a	AC-11
FR-41	Quiz Retry	BR-37, BR-50	quiz_attempts	WF-3.5a	—

13. Constraints

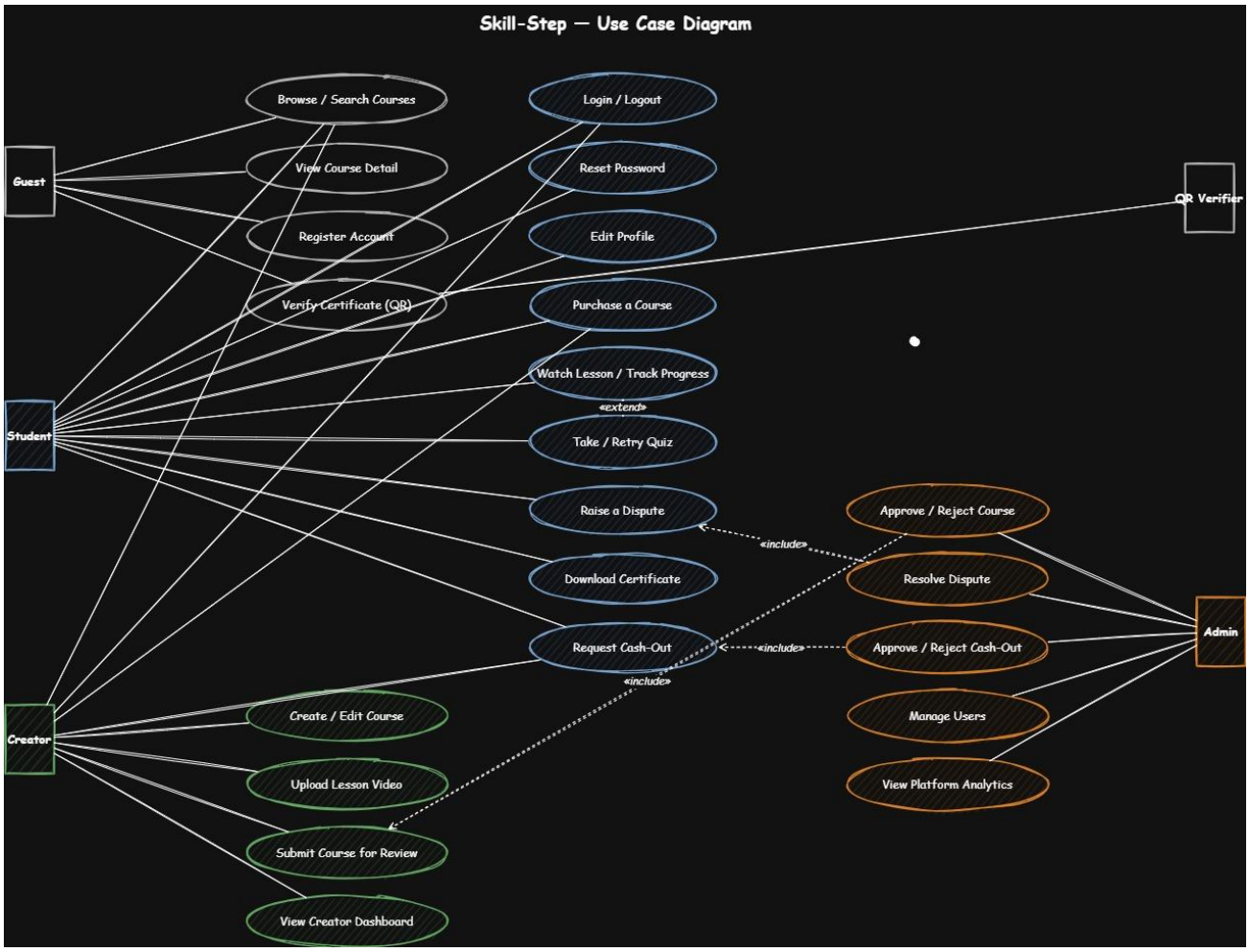
- Implementation must use PHP 8.x, MySQL 8.0+ (InnoDB engine required for row-level locking and ACID guarantees), HTML5, CSS3, and vanilla JavaScript ES6. No server-side JavaScript frameworks.
- All third-party libraries must be open-source. No paid SDK or API keys required for core functionality. Cash-out is manually processed externally.
- The platform must be deployable on a standard LAMP/WAMP single-server stack. Horizontal scaling and CDN delivery are out of scope.
- Development team: 2 undergraduate students, supervised by faculty. Timeline: one academic semester.
- This is a graduation project prototype not required to handle more than 1,000 concurrent users or more than 100 GB of video data.
- No real financial credentials may be used in any demonstration or testing environment. All payment-related fields must use clearly synthetic placeholder values.
- The prototype is not suitable for real commercial deployment without a separate security and compliance audit, particularly regarding financial data handling and privacy regulations.
- MySQL InnoDB engine must be used (not MyISAM or others). InnoDB is required for the row-level locking strategy (SELECT FOR UPDATE) that underpins the concurrency guarantees in Section 3.10.

14. Behavioral and Structural Diagrams

This section presents the complete visual modeling of the Skill-Step platform's functional behavior and system structure. The diagrams collectively describe the dynamic interactions between actors and the system, the flow of data through processes, the state transitions of critical entities, and the sequence of activities in key workflows.

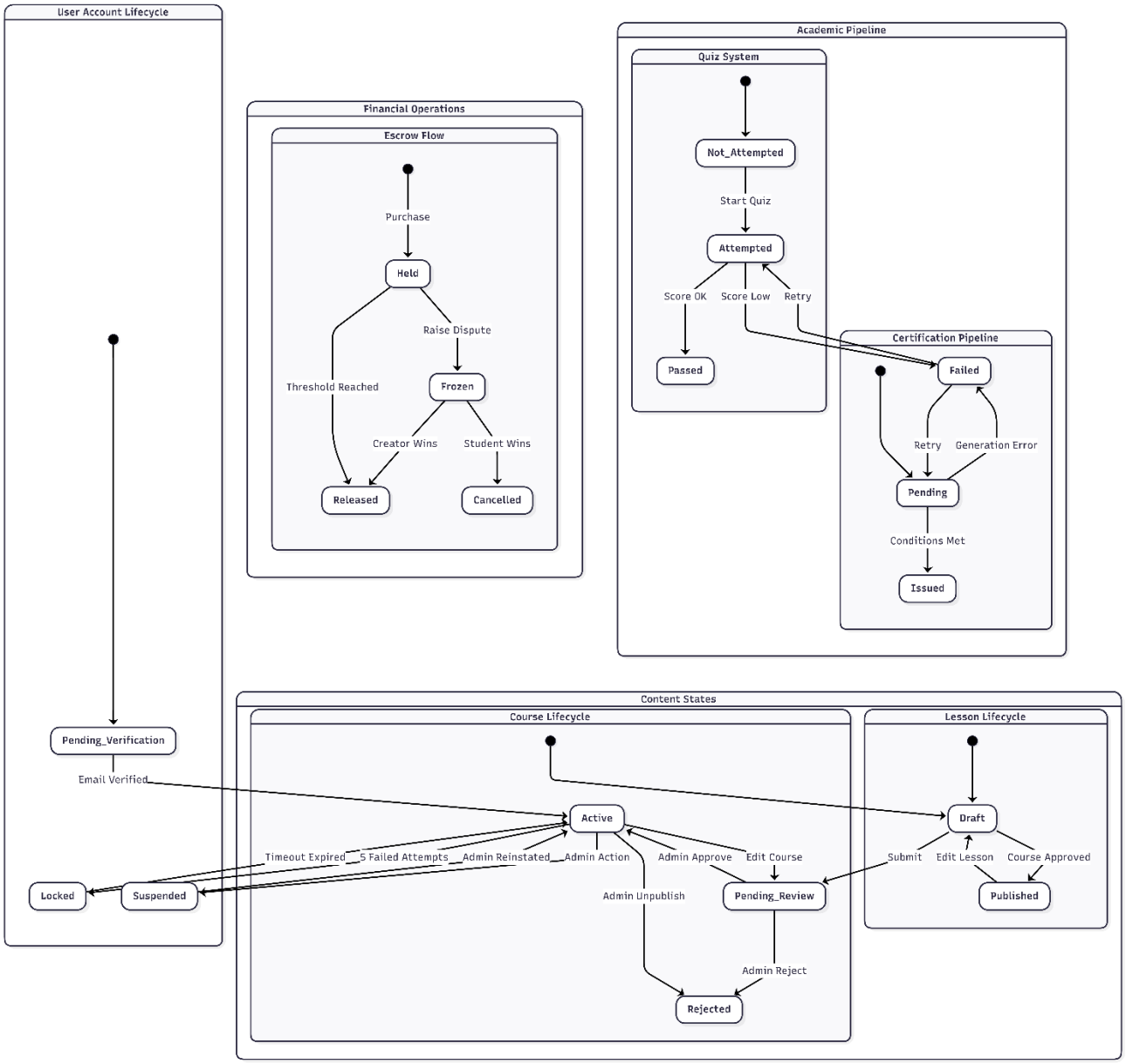
14.1 Use Case Diagrams

To capture functional requirements from an end-user perspective, define system boundaries, and establish clear actor-system responsibilities.



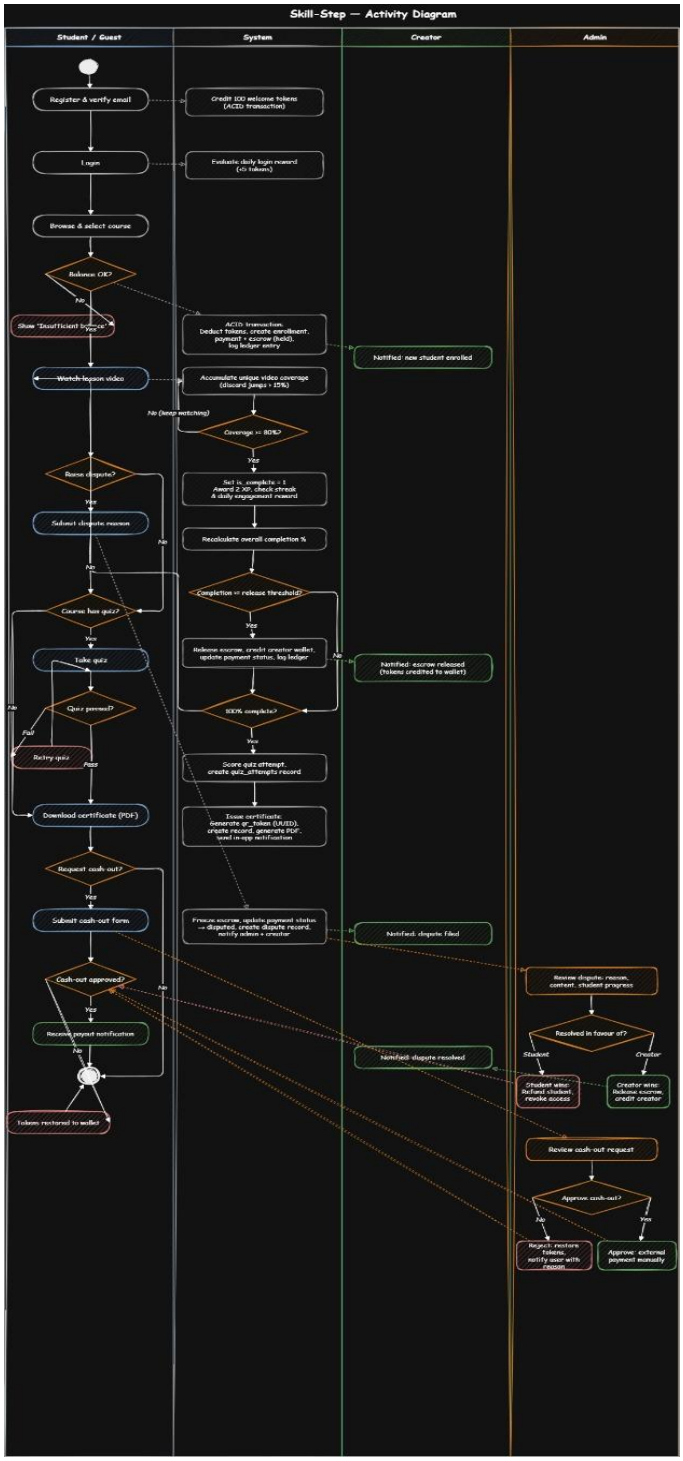
14.2 State Diagram

To model the discrete, event-driven behavior of entities whose responses depend on their current state, ensuring correct state management and preventing illegal transitions.



14.3 Activity Diagrams

To illustrate the dynamic sequence of operations, conditional logic, and concurrent processing within critical system workflows .

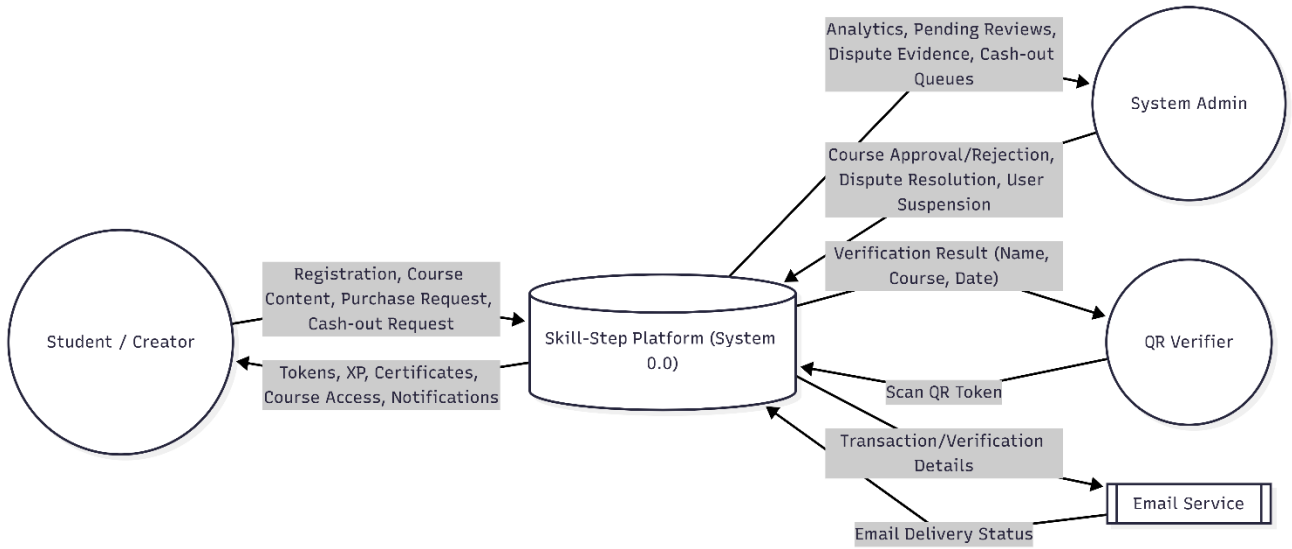


14.4 Data Flow Diagrams (DFD)

Data flow diagrams illustrate how data moves through the system, where it is stored, and which processes transform it. They focus on the inputs, outputs, and data stores rather than control flow.

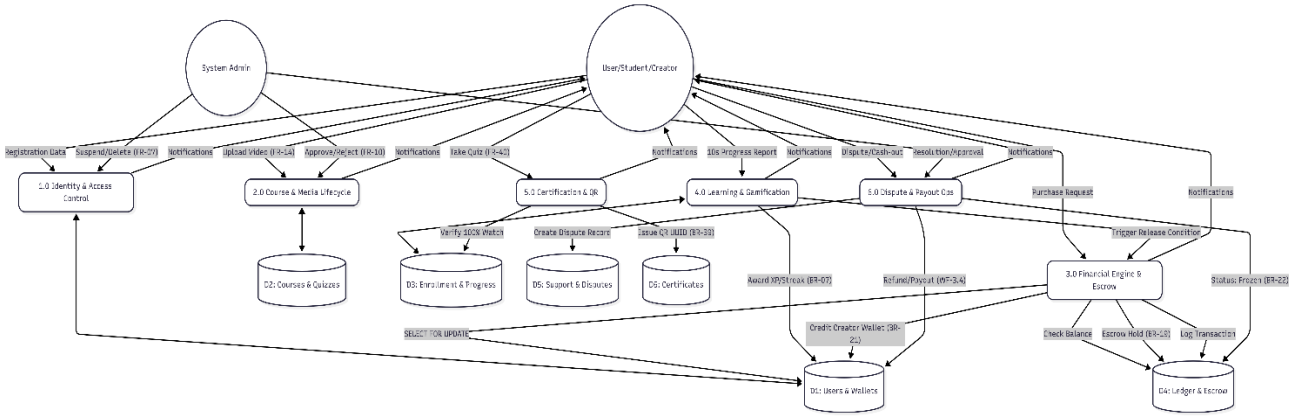
14.4.1 DFD Level 0 — Context Diagram

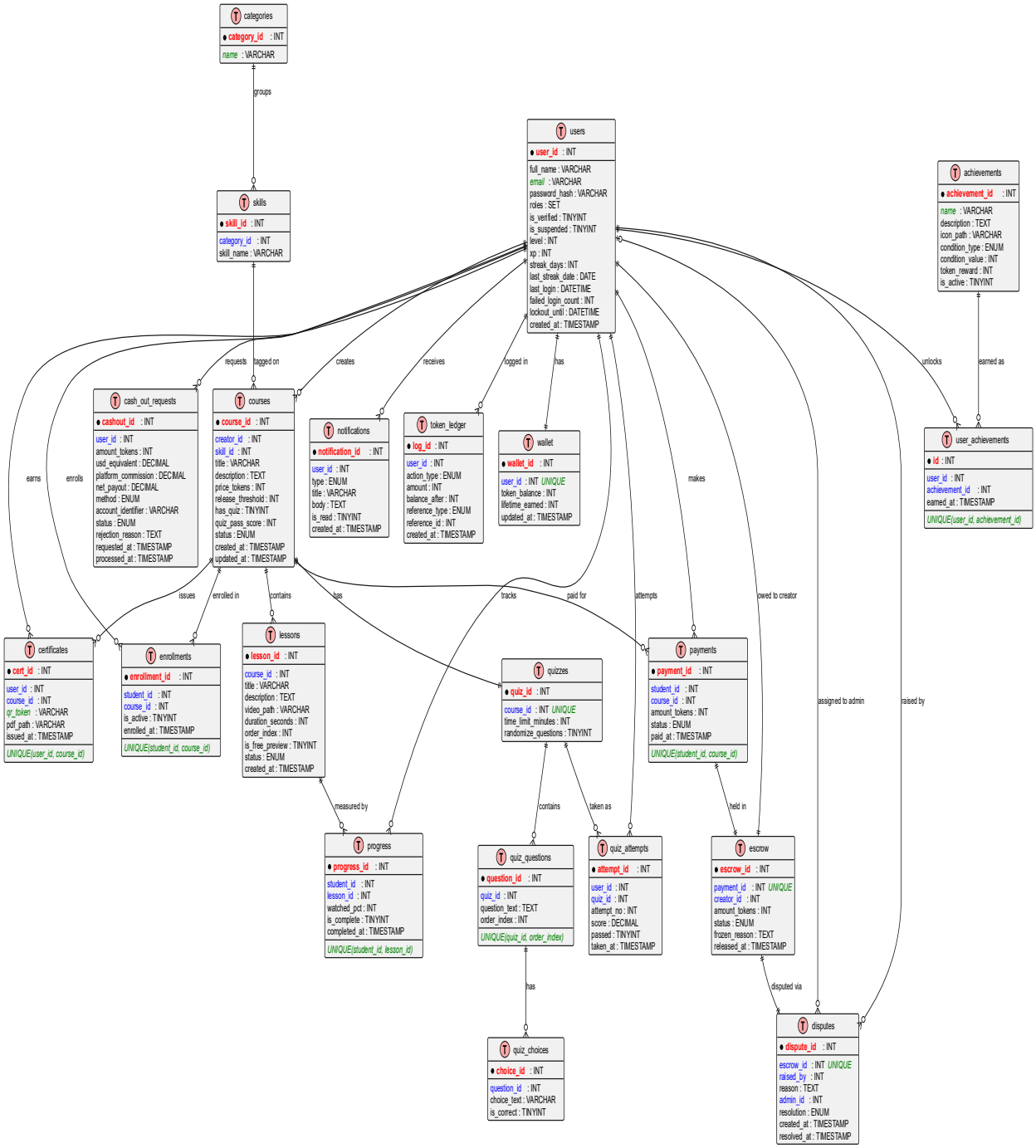
To establish system boundaries, identify all external interactions, and provide a high-level overview of data exchange.



14.4.2 DFD Level 1

To provide a detailed view of system processes, data storage, and internal data movement, serving as a blueprint for database schema design and module implementation.



14.5 Entity Relationship Diagram (ERD)

To provide a structural blueprint of the system's data architecture, defining the entities, their specific attributes, and the logical relationships between them. This section ensures that the database design supports all business rules, normalization standards, and data integrity requirements for the Skill-Step platform.